

CaseWhy — Technical Brief for USCIS Production API Access

Prepared for USCIS Torch Developer Support ahead of the Case Status API production-access demo. Version 3, September 22, 2026.

1. **Primary contact:** as listed on the production-access attestation.
2. **Applicant:** CaseWhy LLC (Florida).
3. **What CaseWhy is:** a free web application that lets people track the status of their own USCIS case by receipt number, explains what a status means in plain language, and shows published processing times and Visa Bulletin data. It is not a law firm and gives no legal advice. Live at app.casewhy.com.
4. **Demo environment:** the live demo runs on a dedicated account using sandbox test receipt numbers only — no real personally identifiable information appears on screen at any point.
5. **In production:** the same code path applies to real receipt numbers, and the app enforces its own internal daily and per-second call caps regardless of whatever quota USCIS ultimately grants (see "Rate limiting and volume").

USCIS Demo Criteria

#	Criterion	Self-assessment	Where it's demonstrated
1	Data entry usability	Ready	Receipt-number field with real-time format feedback, error copy, sign-in, Spanish toggle, mobile-responsive layout.

#	Criterion	Self-assessment	Where it's demonstrated
2	Application data converts to a JSON payload	Ready	A "Show technical details" panel on the on-demand check button renders the exact outbound request (method, URL, headers, redacted token) generated by the same code path that runs the call.
3	OAuth 2.0 authentication	Ready	Client Credentials grant, token cached and refreshed server-side, credentials never reach the browser.
4	HTTPS response handling of all documented response codes, success and error, message shown in the UI	Ready	Every response code the client can encounter routes through one uniform, tested error-handling path. See the response-code table below.
5	Case status tracking	Ready	Add a case, dashboard display, on-demand and scheduled checks, status-history table, change-notification email.

1. A `demo_id` header, assigned by USCIS, is sent on every Case Status API request while the demo is active (`src/lib/uscis/client.ts`, `demoIdHeader()`).
2. File upload / encoded-document handling is a FOIA API criterion, not a Case Status API one, and is out of scope for this demo.

1. Data entry usability

1. The receipt-number field gives real-time format feedback as the user types (3 letters followed by 10 digits), with clear error copy.¹

2. Submission is never blocked on format alone — a well-formed number still might not correspond to a real case.
3. The app supports English and Spanish throughout.
4. The app is fully responsive on mobile.
5. A new user can sign up and track their first case in under two minutes.

2. JSON payload conversion

1. `getCaseStatus()` builds the outbound request to USCIS's Case Status API from the entered receipt number.²
2. The exact request that will be sent — method, URL, and headers, with the access token redacted — is visible in the dashboard via a "Show technical details" toggle on the on-demand check button.³
3. That preview is generated by the same function that makes the real call, not a separate description.

3. OAuth 2.0 authentication

1. CaseWhy uses the OAuth 2.0 Client Credentials grant.⁴
2. `getAccessToken()` requests a token from USCIS's token endpoint using the client ID and secret, and caches it in memory, refreshing automatically shortly before expiry rather than on every call.⁵
3. Sandbox and production use separate credential pairs and separate token/API base URLs, selected by a single environment variable — there is no code path where they can be mixed up.⁶
4. Credentials are read from environment variables, never committed to source control, and live only as encrypted environment variables in Vercel.
5. The USCIS API client runs exclusively on the server.
6. **Client ID, client secret, and access tokens never reach the browser** — the browser only ever talks to CaseWhy's own server, which in turn talks to USCIS.⁷

4. HTTPS response handling — every documented response code

1. Every response from USCIS's API — success or error — is handled by the same code path: `uscisRequest()` → `UscisApiError` → `extractUscisErrorMessage()`.⁸
2. That message reaches the interface directly.
3. There is no automatic retry logic and no per-code special-casing that could silently diverge from this path.

Code	Meaning	Tested how	What the user sees
200	Success	Live, routinely (see "Sandbox testing")	Status text, description, and history rendered in the dashboard.
422	Malformed receipt number	Live, real sandbox response	USCIS's own message, shown directly (confirmed: <i>"The application receipt number is not formatted correctly..."</i>).
404	Unrecognized receipt number	Live, real sandbox response	USCIS's own message, shown directly (confirmed: <i>"Case Status Online does not recognize the receipt number entered..."</i>).

Code	Meaning	Tested how	What the user sees
429	Rate limit exceeded	Live — occurred once during the sandbox volume test (one call out of 50 in a single batch, Sep 15)	USCIS's own message, shown directly through the same path as 422/404; the very next scheduled batch completed cleanly. Now additionally guarded against before the call is even made — see "Rate limiting and volume."
401	Authentication failure	Same code path as every other non-2xx response; not independently triggered against this project's own live credentials, since deliberately invalidating a working credential to force a real 401 risks disrupting the in-progress sandbox test window	Same uniform error-message display.
403	Forbidden	Not documented as a Case Status response for this integration; same uniform path would handle it if USCIS ever returns one	Same uniform error-message display.

Code	Meaning	Tested how	What the user sees
500 / 503	Server error	Not independently reproducible against a live third-party system; same uniform path applies	Same uniform error-message display.
Timeout / no response	Network failure or a hung request	A 15-second client-side timeout was added specifically for this demo (previously, a request had no timeout at all) — code-reviewed, not yet observed live since USCIS's sandbox has not hung during testing	Same uniform error-message display: "Request to USCIS timed out after 15 seconds."

4. Receipt numbers are never written to logs in plaintext, for any response code.

5. Case status tracking

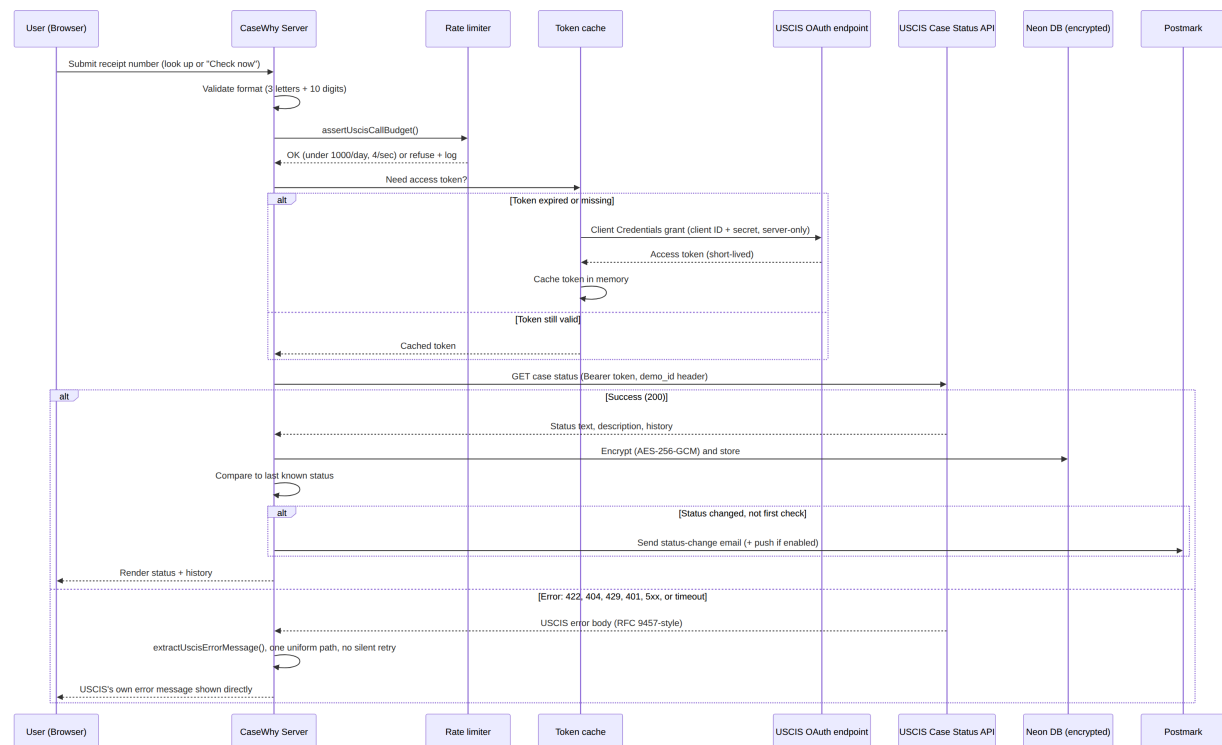
1. A tracked case stores an encrypted receipt number, case type, and last known status.⁹
2. `case_status_history` stores one row per USCIS history entry (service-center prefix only, never the full receipt number).⁹
3. The scheduled status-check job compares the freshly fetched status against the last known one.¹⁰
4. Only on a genuine change, never on the first check, the job sends a Postmark email and, if enabled, a push notification.
5. Adding a case is a short signed-in flow.
6. Removing one is immediate and self-serve (`untrackCase()`), a real, ownership-checked delete).¹¹

- Free accounts get 3 receipt numbers for the life of the account — tracking one or just looking up its status both count against that cap, and untracking a case doesn't free a slot back up.¹²
- The paid tier raises this in tiers up to a hard ceiling, with no lifetime limit.

Request/response transaction flow

The diagram below traces one case-status transaction end to end:

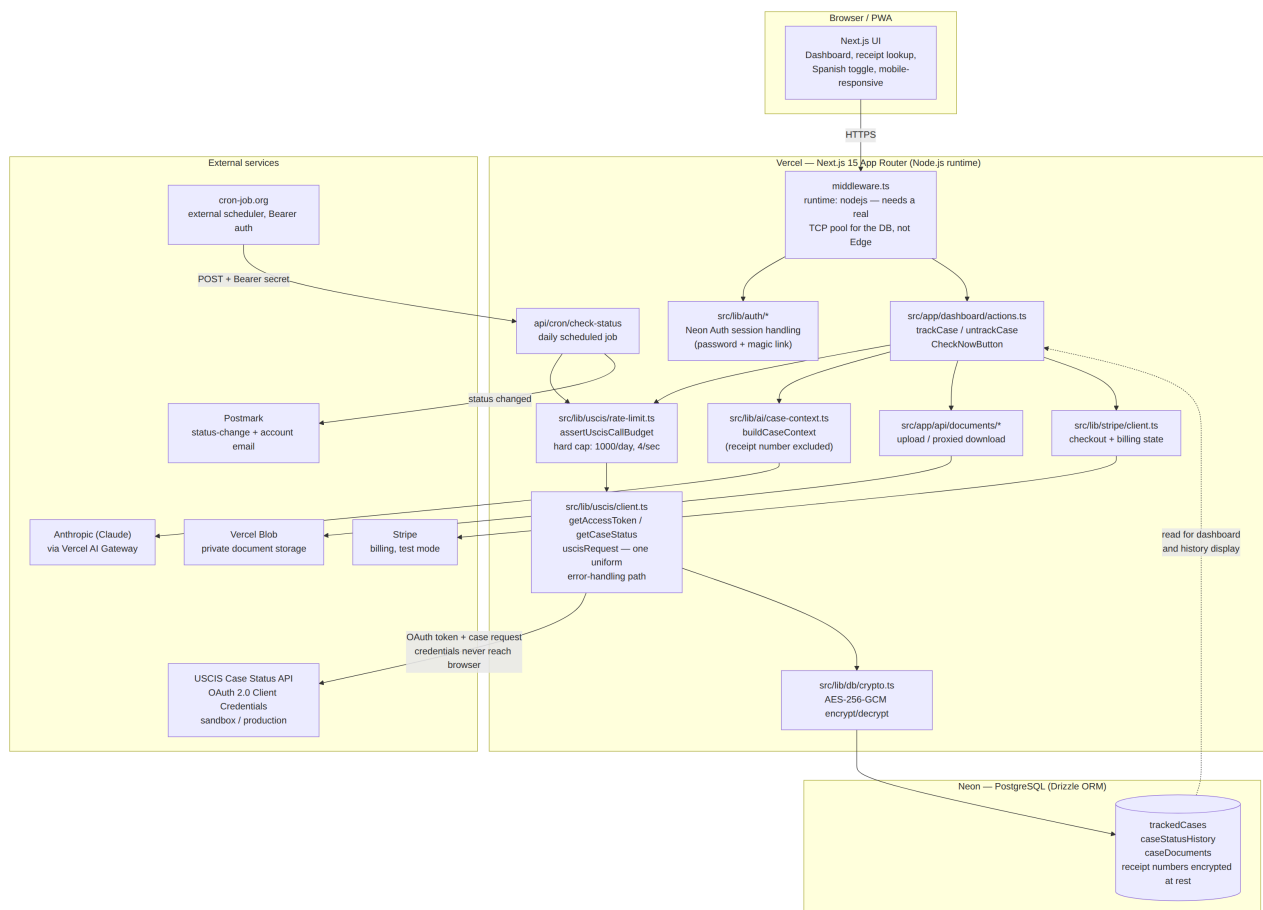
- User submits a receipt number.
- Server validates the format and checks the rate-limit budget.
- Server gets or refreshes a cached OAuth token.
- Server calls the USCIS Case Status API.
- On success: server stores and renders the result, and sends a notification only on a real status change.
- On error (422/404/429/401/5xx/timeout): USCIS's own error message is shown directly.



Additional Technical & Operational Detail

Architecture and hosting

1. CaseWhy is a Next.js 15 application (App Router) hosted on Vercel, with PostgreSQL (Neon) for data storage via Drizzle ORM.¹³
2. Transactional email runs through Postmark.
3. Billing (for the optional paid tier) runs through Stripe.
4. Every server route runs on Vercel's standard Node.js runtime, chosen specifically because the app's database access needs a real TCP connection pool, which the Edge runtime doesn't support.¹⁴
5. There is no native mobile app; CaseWhy is an installable Progressive Web App.



File upload (document vault)

Not a Case Status API criterion — a FOIA API criterion, out of scope for this demo, included here for completeness.

1. The paid tier includes a working document-upload feature: PDF, JPEG, or PNG up to 10 MB.
2. Files are stored in Vercel Blob under private access, never a public URL, with the original filename encrypted before storage.¹⁵
3. Downloads are proxied through CaseWhy's own server rather than exposing a direct Blob URL.¹⁶
4. No uploaded document is ever sent to USCIS or any third party.

Rate limiting and volume

1. The client enforces a real, hard cap before any call is made: no more than 1,000 Case Status API calls per calendar day and no more than 4 in any rolling one-second window.¹⁷
2. Each request is checked against a call log before it's made, and refused (with a logged event) if either limit would be exceeded.
3. These thresholds are set at the sandbox's own confirmed limits (5 TPS, 1,000/day) rather than an unverified production number.
4. USCIS's own production quota will be shared once access is granted, and the caps are configurable to raise once that figure is known.
5. On top of that hard floor, the schedule paces volume naturally: the production status-check job runs on a fixed schedule per tracked case.
6. An on-demand check is a paid-tier feature, rate-limited by real human clicks.
7. Volume scales linearly and predictably with the number of tracked cases, not with traffic spikes.
8. Expected volume at launch: on the order of tens of calls per day (one scheduled check per active tracked case); growth tracks user adoption directly, with the hard cap above as the actual ceiling regardless of how many users sign up.

9. A separate example of the team's general rate-limiting practice: the anonymous "ask a question" feature enforces a durable, database-backed lifetime cap per IP address.¹⁸

Privacy, security, and data handling

1. Receipt numbers are treated as personally identifiable information: encrypted with AES-256-GCM before being written to the database, with the encryption key held separately from the database itself.¹⁹
2. No CaseWhy staff member can view a receipt number in plaintext through any administrative tool.
3. What is sent to the AI service (Claude, via Anthropic, through the Vercel AI Gateway): form type, processing center, dates, status text/description, full status history, and matched policy background.
4. **The raw receipt number is never included**, by design in the prompt-construction code.²⁰
5. The one exception: the paid-tier escalation-letter drafting tool includes the receipt number in the drafted letter itself, since the letter has to identify the case to USCIS or a congressional office; this exception is disclosed in the Privacy Policy alongside the general rule.²¹
6. Third parties that receive any case- or account-related data:
 1. **Anthropic** — status text, dates, and history for AI explanations; additionally the receipt number when drafting an escalation letter, as above.
 2. **Postmark** — the notification email address and status text, to send account and status-change emails.
 3. **Vercel and Neon** — infrastructure (hosting and database), with no independent access to decrypted data outside the app's own code.
 4. **Stripe** — billing information for the optional paid tier only, not case data.
7. Each third party is contractually and/or technically limited to using data only for the relevant part of the service.²²
8. Authentication is via Neon Auth, supporting both password and magic-link sign-in.²³

9. Admin access is gated by a single allow-listed email, fail-closed if unset.²⁴
10. Removing an individual tracked case is immediate and self-serve.
11. Full account deletion is a request to `privacy@casewhy.com`, completed within 30 days.²⁵
12. Users are required to actively acknowledge a material change to the Privacy Policy or Terms of Use the next time they sign in, not just notified by email.²⁶
13. Security headers — HSTS, a Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, and Referrer-Policy — are set on every response.²⁷
14. Automated dependency-update monitoring runs via Dependabot, with `npm audit` also surfaced in CI.
15. Two pre-existing transitive dev-tooling findings (not runtime-reachable application code) are open pending a deliberate major-version upgrade, tracked rather than silently ignored.²⁸

Business and operations

1. Free tier: 3 receipt numbers for the life of the account, sign-in required to look one up, full access to all reference data.²⁹
2. Free tier also includes a separate, always-free anonymous policy-question tool with its own lifetime cap per visitor, no account needed.
3. Paid "Plus" tier: \$9.99/month, \$39.99/6 months, or \$69.99/year.
4. **Nothing from the USCIS Case Status API is ever paywalled** — status checking is free at every tier.
5. Plus adds: more tracked cases, unlimited AI questions, on-demand checks, the document vault, and drafting tools.
6. Billing runs on Stripe, currently in test mode — CaseWhy has processed no real production payments to date.³⁰
7. Support runs through 13 real, monitored email aliases on the domain, including `security@`, `privacy@`, `legal@`, and `abuse@`.³¹
8. If production API access were ever revoked, CaseWhy has no scraping fallback and would show an honest degraded-service message rather than

attempt an unauthorized alternative path to USCIS data.

Sandbox testing

1. CaseWhy ran a dedicated 5-business-day sandbox volume test, September 14–18, 2026, using the app's real production code path, not a mock.³²
 2. Each run drew from a confirmed pool of real sandbox test receipt numbers spanning 7 form types.
 3. Roughly 50 calls per run (a mix of successful and deliberate-error lookups), 18 runs/day across the sandbox's 7 AM–8 PM ET operating hours.
 4. That's roughly 900 calls/day, under the sandbox's published 1,000-request/day, 5-transactions-per-second ceiling.
 5. Monday (Sep 14): ran clean throughout.
 6. Tuesday (Sep 15): 15 runs, ~750 calls, with one batch returning a single real 429 (rate-limit) response — the very next scheduled batch completed with no issue, and no other run that day or since repeated it.
 7. Wednesday (Sep 16): 11 runs, ~550 calls, cleanly, at time of the original writing.
 8. Thursday–Friday (Sep 17–18): completed the test window.
-

1. `src/lib/uscis/receipt-number.ts`;
`src/app/dashboard/ReceiptNumberInput.tsx`. ↩
2. `src/lib/uscis/client.ts`, `getCaseStatus()`. ↩
3. `src/lib/uscis/client.ts`, `describeCaseStatusRequest()`;
`src/app/dashboard/CheckNowButton.tsx`. ↩
4. `src/lib/uscis/client.ts`, `getAccessToken()`. ↩
5. `src/lib/uscis/client.ts`, in-memory token cache keyed by client ID. ↩
6. `src/lib/config.ts`, `getUscisConfig()`, `USCIS_ENV`. ↩
7. `src/lib/uscis/client.ts`; `src/middleware.ts` (runtime:
"nodejs"). ↩
8. `src/lib/uscis/client.ts`, `uscisRequest()`, `UscisApiError`,
`extractUscisErrorMessage()`. ↩

9. `src/lib/db/schema.ts`, `trackedCases` and `caseStatusHistory` tables. [↩↩](#)
10. `src/lib/uscis/check-status.ts`, `checkTrackedCaseNow()`. [↩](#)
11. `src/app/dashboard/actions.ts`, `trackCase()`, `untrackCase()`. [↩](#)
12. `src/lib/billing/tier.ts`, `TIER_LIMITS`, `PLUS_HARD_CEILING_MAX_CASES`. [↩](#)
13. `package.json`; `src/lib/db/client.ts`. [↩](#)
14. `src/middleware.ts`; `next.config.ts` (`experimental.nodeMiddleware`). [↩](#)
15. `src/app/api/documents/route.ts`; `src/lib/db/schema.ts`, `caseDocuments` table. [↩](#)
16. `src/app/api/documents/[id]/route.ts`. [↩](#)
17. `src/lib/uscis/rate-limit.ts`, `assertUscisCallBudget()`. [↩](#)
18. `src/lib/get-help/anonymous-usage.ts`, `LIFETIME_CAP_PER_IP`. [↩](#)
19. `src/lib/db/crypto.ts`. [↩](#)
20. `src/lib/ai/case-context.ts`, `buildCaseContext()`. [↩](#)
21. Privacy Policy, Section 2, `www.casewhy.com/privacy.html`. [↩](#)
22. Privacy Policy, Section 5, `www.casewhy.com/privacy.html`. [↩](#)
23. `src/lib/auth/server.ts`; `src/app/auth/sign-in/page.tsx`. [↩](#)
24. `src/lib/auth/admin.ts`, `isAdminEmail()`. [↩](#)
25. Privacy Policy, Section 7, `www.casewhy.com/privacy.html`. [↩](#)
26. `src/lib/policy/versions.ts`; the `acknowledgment-gate` middleware. [↩](#)
27. `next.config.ts`, `headers()`. [↩](#)
28. `.github/dependabot.yml`; `.github/workflows/ci.yml`. [↩](#)
29. `src/lib/billing/pricing.ts`, `getAllEffectivePrices()`. [↩](#)
30. `src/lib/stripe/client.ts`. [↩](#)
31. `scripts/seed-email-alias-configs.ts`; `email_alias_configs` table. [↩](#)
32. `src/app/api/cron/uscis-volume-test/route.ts`; `src/lib/uscis/sandbox-volume-test.ts`. [↩](#)